



Presentación 1

TOMAS SILVA
JUAN PABÓN VARGAS
ALEXANDER APONTE



CONTENIDO



Estilo Arquitectonico



Frontend



Backend



Base de Datos



Servivio de Integración



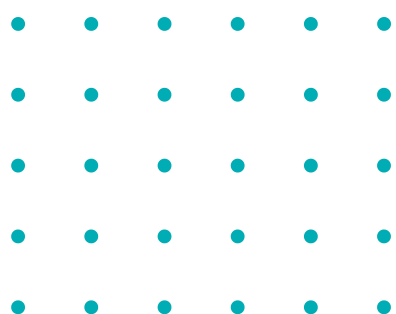
Diseño de propuesta



Implementación



Lecciones aprendidas



LAYERED ARCHITECTURE

Patrón de diseño que organiza el código de una aplicación en capas con roles definidos.

UI - BBL - DAL

(User Interface - Business Logic Layer - Data Access Layer)

- **N-Tiers**

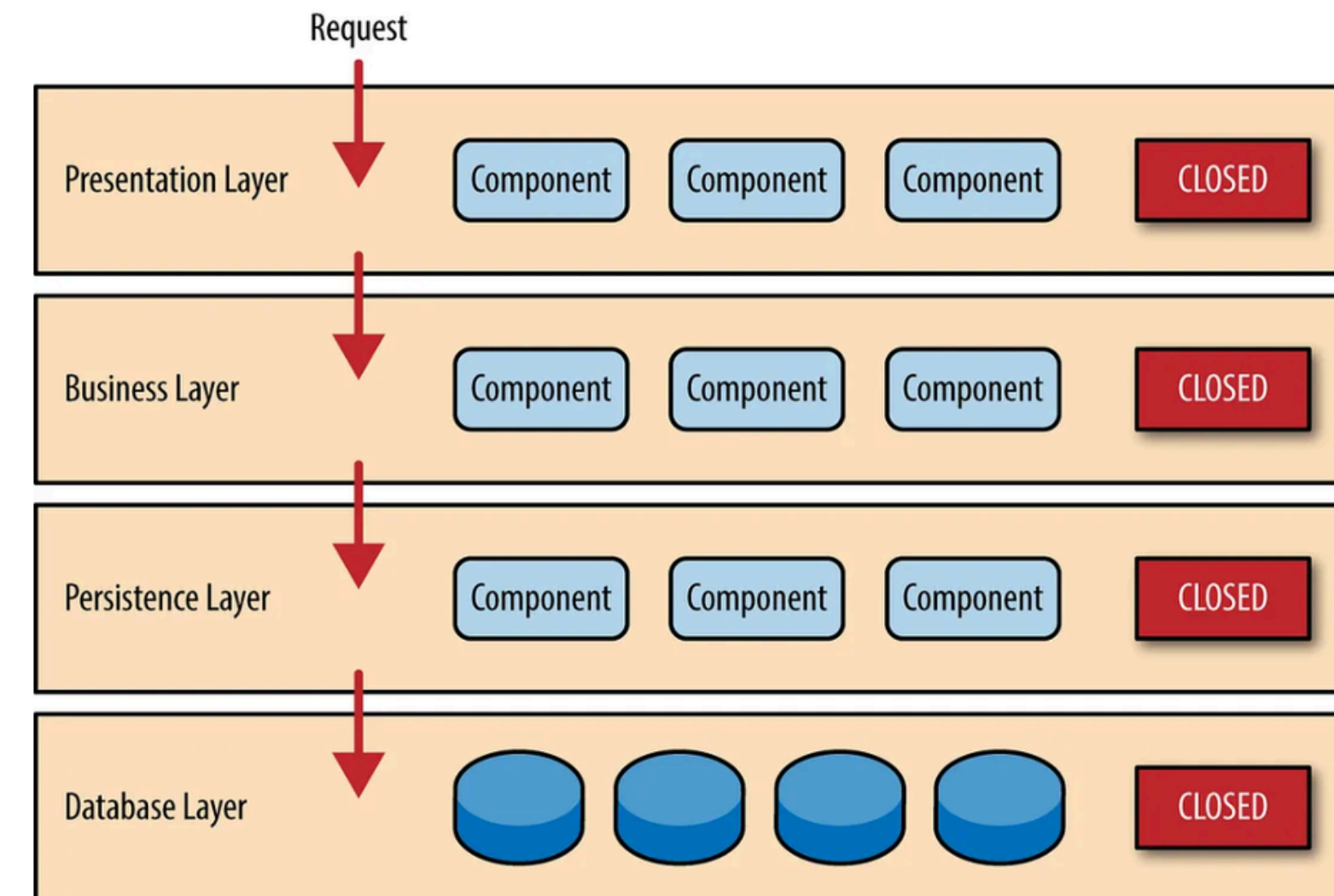
- Organiza el sistema en un número definido de capas.
- Cada capa tiene una responsabilidad específica.
- La cantidad de capas depende de las necesidades del sistema.
- Las capas pueden correr en diferentes maquinas

- **Local o Distribuido**

- Local: todas las capas funcionan en el mismo entorno.
- Distribuido: las capas funcionan en diferentes procesos o servidores.
- Define cómo se despliega el sistema.

- **Capas Cerradas vs. Abiertas**

- Cerradas: una capa se comunica directamente solo con la siguiente.
- Abiertas: una capa puede comunicarse directamente con capas inferiores.
- Cerradas = mayor control; abiertas = mayor flexibilidad.



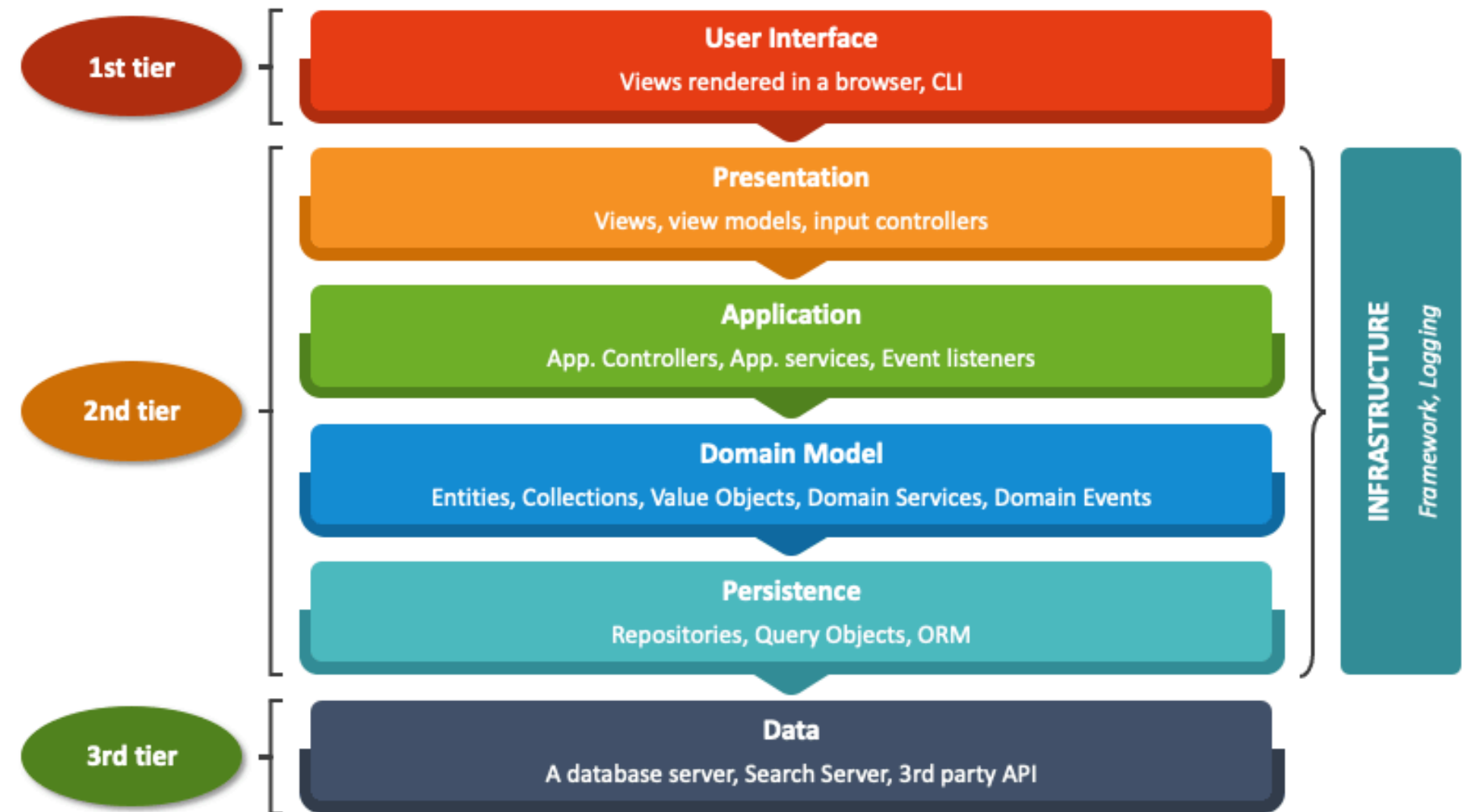
Sistema Distribuido

LAYERED ARCHITECTURE

CARACTERISTICAS

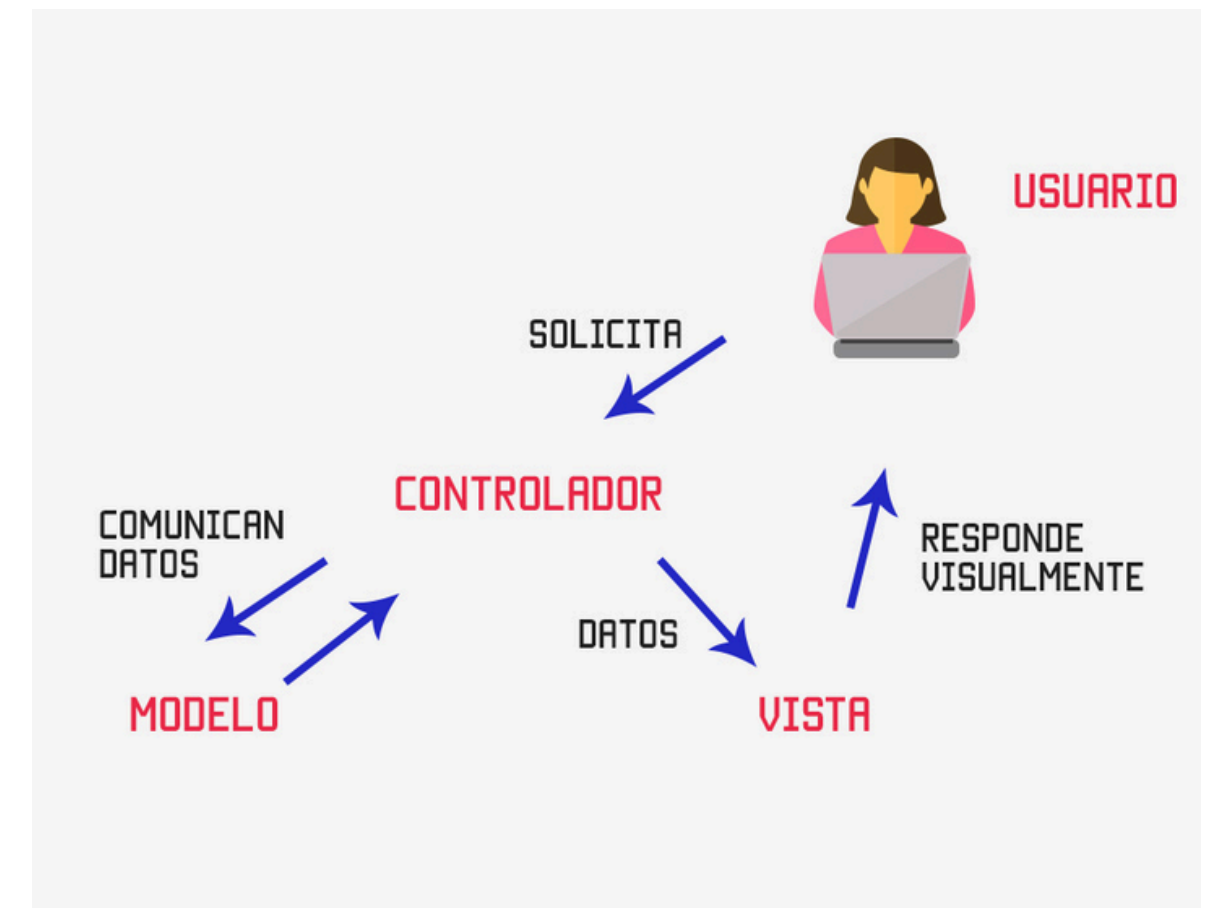
- Separación de responsabilidades
- Niveles de abstracción
- Dependencia entre capas
- Encapsulamiento
- Bajo acoplamiento
- Alta cohesión
- Comunicación controlada
- Independencia de implementación
- Flexibilidad de despliegue
- Escalabilidad y mantenibilidad

LAYERED ARCHITECTURE



PATRONES APLICABLES

- **MVC:** separa presentación, control y modelo para organizar la interfaz y la lógica.
- **Service Layer:** concentra los casos de uso y operaciones de negocio.
- **Repository:** abstrae el acceso a los datos y separa la persistencia de la lógica.
- **Dependency Injection:** proporciona dependencias externamente para reducir acoplamiento.
- **Data Mapper / ORM:** relaciona los objetos de la aplicación con los datos de la base de datos.
- **DAO:** encapsula las operaciones de acceso y manipulación de datos.
- **Unit of Work:** agrupa varias operaciones de persistencia en una unidad.
- **Active Record:** combina una entidad con sus operaciones de persistencia.
- **Facade:** simplifica el acceso a funcionalidades de varios componentes.
- **Adapter:** adapta interfaces incompatibles para permitir su comunicación.
- **Proxy:** controla el acceso a otro componente mediante un intermediario.
- **Retry:** repite operaciones que fallan temporalmente.
- **Timeout:** limita el tiempo de espera de una operación.
- **Health Checks:** comprueba el estado y disponibilidad de los componentes.



VENTAJAS Y DESVENTAJAS



VENTAJAS



Separación de responsabilidades



Fácil de desarrollar



Fácil de probar



Fácil de mantener



Seguridad por aislamiento de servidores en subredes



DESVENTAJAS



Rendimiento



Escalabilidad



Complejidad de despliegue



Anclado a un Stack tecnológico



Tolerancia a los fallos (fallo en cascada)

PROBLEMAS COMUNES

Causados generalmente al
implementar ignorando
ciertas características

- Acoplamiento excesivo: una capa depende demasiado de los detalles internos de otra.
- Layer Skipping: una capa accede directamente a otra capa inferior, ignorando las capas intermedias.
- Capas demasiado grandes: una capa termina concentrando demasiadas responsabilidades y se vuelve difícil de mantener.
- Duplicación de lógica: las mismas reglas o validaciones aparecen en diferentes capas.
- Lógica de negocio en la presentación: las reglas del negocio se implementan en controladores o interfaces en lugar de estar centralizadas en la capa correspondiente.
- Lógica de negocio en persistencia: los repositorios o componentes de acceso a datos comienzan a encargarse de reglas que deberían pertenecer al negocio.
- Exceso de comunicación entre capas: una operación sencilla debe atravesar muchas capas, generando código innecesario y, en arquitecturas distribuidas, mayor latencia.
- Dependencias rígidas: una modificación en una capa provoca cambios en varias capas superiores o inferiores.



CASOS DE USO Y APLICACIÓN

Usar cuando:

- El sistema tiene responsabilidades claramente diferenciadas.
- Se necesita una arquitectura sencilla de comprender.
- El equipo conoce bien el modelo.
- Existe una aplicación empresarial tradicional.
- Se necesita separar UI, negocio y persistencia.
- El sistema no requiere una arquitectura altamente especializada.
- Se quiere una base sencilla para comenzar un proyecto.

No usar cuando:

- Sistemas con alta necesidad de escalabilidad independiente.
- Sistemas altamente orientados a eventos.
- Sistemas con dominio extremadamente complejo.
- Sistemas con requisitos extremos de rendimiento.

- Sistemas bancarios
- Comercio electrónico
- Sistemas empresariales (ERP/CRM)
- Sistemas universitarios
- Aplicaciones web
- Aplicaciones empresariales con Java o .NET



FRONTEND

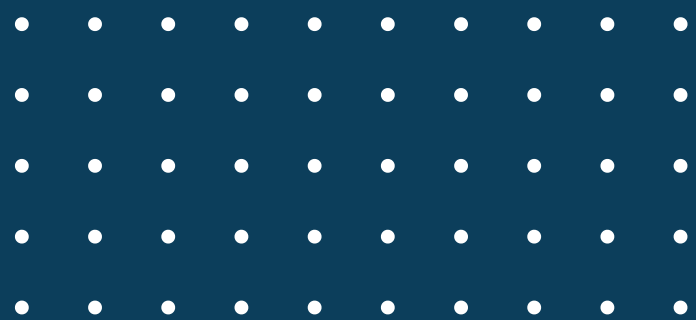
ANGULAR - JS



- Framework de desarrollo web utilizado principalmente para crear aplicaciones web dinámicas y de una sola página.
- Un framework basado en componentes que proporciona herramientas para organizar la interfaz, gestionar datos, navegación, formularios y comunicación con servicios.
- Angular utiliza principalmente TypeScript, que se compila a JavaScript para ejecutarse en el navegador.

CARACTERÍSTICAS PRINCIPALES

- Arquitectura basada en componentes: divide la aplicación en componentes reutilizables que gestionan diferentes partes de la interfaz.
- Data binding: permite sincronizar los datos de la aplicación con los elementos de la interfaz.
- Inyección de dependencias: facilita la creación y reutilización de servicios y componentes.
- Routing: permite gestionar la navegación entre diferentes vistas de una aplicación web.
- Formularios: proporciona herramientas para crear y validar formularios.
- Programación reactiva: permite gestionar cambios y flujos de datos de forma dinámica.
- Angular CLI: proporciona herramientas para crear, configurar, desarrollar y compilar aplicaciones.
- Aplicaciones de una sola página (SPA): permite crear aplicaciones donde diferentes vistas pueden cargarse sin realizar una recarga completa de la página.
- Modularidad: permite organizar las funcionalidades de una aplicación en diferentes componentes y módulos para facilitar su mantenimiento.



VENTAJAS Y DESVENTAJAS



VENTAJAS



Proporciona una estructura clara para proyectos grandes.



Incluye muchas funcionalidades directamente en el framework.



Facilita la reutilización mediante componentes y servicios.



Cuenta con herramientas oficiales de desarrollo y soporte de Google.



DESVENTAJAS



Tiene una curva de aprendizaje relativamente alta.



Puede resultar excesivo para proyectos pequeños.



Tiene un ecosistema amplio que requiere mantener compatibilidades entre versiones.



Su estructura puede generar más código y configuración que alternativas más simples.

CASOS DE USO

- Se desarrollan aplicaciones web medianas o grandes.
- Existen múltiples vistas, formularios y funcionalidades.
- Se necesita una estructura organizada para equipos de desarrollo.
- Se desarrollan sistemas empresariales, administrativos o dashboards.
- Se requiere una SPA con bastante lógica del lado del cliente.

No usar cuando:

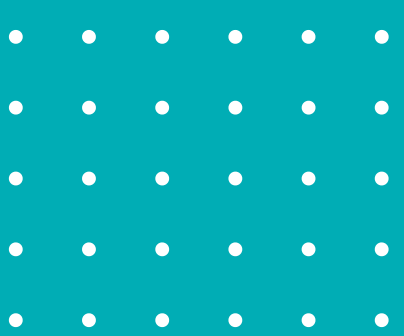
- Solo se necesita una página estática sencilla.
- El proyecto tiene muy poca lógica e interacción.
- Se busca minimizar al máximo las dependencias y abstracciones.



CASOS DE APLICACIÓN

- Sistemas administrativos.
- Plataformas financieras.
- Sistemas educativos.
- Dashboards empresariales.
- Plataformas SaaS.
- Portales corporativos.
- Sistemas de gestión.





QUE TAN COMÚN ES EL STACK

- Angular/JS es ampliamente utilizado para el desarrollo de aplicaciones web, especialmente en aplicaciones empresariales y proyectos de tamaño mediano o grande.
- Angular funciona como el framework principal para estructurar y desarrollar la aplicación frontend, mientras que JS proporciona el lenguaje utilizado para ejecutar la lógica en el navegador.
- Angular proporciona herramientas y estructuras para organizar la aplicación, mientras que JavaScript permite ejecutar el comportamiento y la lógica del lado del cliente.
- Además, Angular puede utilizar otras tecnologías complementarias, como HTML y CSS para la estructura y presentación de la interfaz, y Node.js y npm como herramientas del entorno de desarrollo y administración de paquetes.



BACKEND

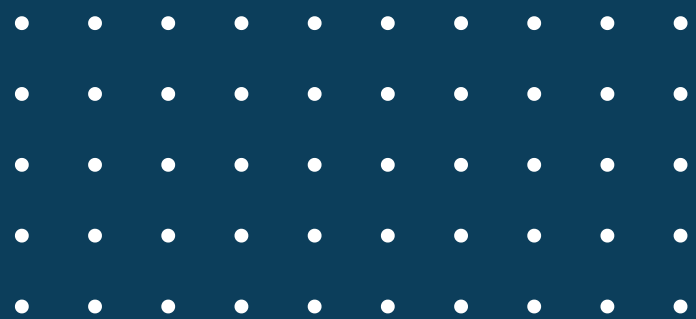
FAST API/PYTHON

- Framework web de Python orientado principalmente al desarrollo de APIs y servicios web.
- Herramienta para construir backends que reciben solicitudes, procesan información y devuelven respuestas, normalmente mediante HTTP.
- Framework basado en estándares como OpenAPI y JSON Schema, que permite generar documentación de la API automáticamente.



CARACTERÍSTICAS PRINCIPALES

- Desarrollo de APIs: permite crear endpoints para recibir y responder solicitudes HTTP.
- Soporte asíncrono: permite utilizar operaciones asíncronas mediante `async` y `await`.
- Validación de datos: utiliza las declaraciones de tipos y modelos para validar los datos recibidos.
- Documentación automática: genera documentación interactiva basada en OpenAPI.
- JSON: permite recibir y devolver datos en formatos compatibles con JSON.
- Inyección de dependencias: proporciona un sistema para gestionar dependencias entre componentes.
- Integración con Python: aprovecha las características y bibliotecas del ecosistema Python.



VENTAJAS Y DESVENTAJAS



VENTAJAS



Permite desarrollar APIs rápidamente.



Tiene validación y documentación integradas.



Soporta programación asíncrona.



Se integra bien con otras herramientas del ecosistema Python.



Es adecuado para servicios que necesitan comunicarse mediante APIs.



DESVENTAJAS



Requiere conocimientos de Python y conceptos de desarrollo de APIs.



Algunas aplicaciones pueden requerir configurar herramientas adicionales para autenticación, persistencia o tareas específicas.



El desarrollo asíncrono puede aumentar la complejidad cuando no se conocen bien sus conceptos.

CASOS DE USO

- Los datos tienen estructuras variables.
- Se necesita evolucionar frecuentemente el modelo de datos.
- Los datos se pueden representar naturalmente como documentos.
- Se trabaja con catálogos de productos o contenido.
- Se desarrollan aplicaciones web, móviles o sistemas distribuidos.
- Se necesita almacenar grandes cantidades de documentos.

No usar cuando:

- El proyecto solamente necesita una página web estática.
- No se necesita un backend.
- Se requiere principalmente una aplicación de escritorio sin comunicación mediante servicios web.



CASOS DE APLICACIÓN

- APIs empresariales.
- Servicios de aprendizaje automático.
- Sistemas de procesamiento de datos.
- Backends para aplicaciones web.
- Microservicios.
- Servicios internos.

QUE TAN COMÚN ES EL STACK

Python + FastAPI es un stack bastante utilizado dentro del desarrollo web con Python. Python proporciona el lenguaje, FastAPI proporciona la estructura para desarrollar el backend y la API utiliza HTTP y formatos como JSON para comunicarse con otras aplicaciones.



BASE DE DATOS

MONGODB/DOCUMENT

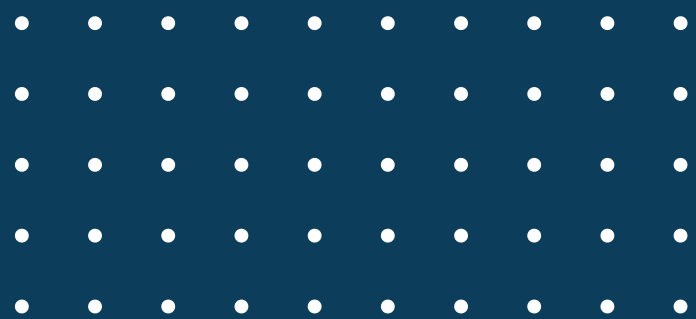


mongo DB

- Base de datos NoSQL orientada a documentos.
- Almacena la información en documentos BSON organizados dentro de colecciones.
- Su modelo permite representar los datos mediante estructuras similares a objetos y documentos JSON.

CARACTERÍSTICAS PRINCIPALES

- Modelo orientado a documentos: la información se almacena en documentos.
- BSON: utiliza una representación binaria de documentos similares a JSON.
- Esquema flexible: los documentos de una colección no necesitan tener exactamente los mismos campos.
- Documentos embebidos: permite almacenar información relacionada dentro de un mismo documento.
- Referencias: también permite relacionar documentos almacenados en diferentes colecciones.
- Índices: permite crear índices para mejorar el rendimiento de las consultas.
- Escalabilidad: está diseñado para trabajar con grandes cantidades de datos y cargas distribuidas.
- Validación de esquemas: aunque el modelo es flexible, pueden establecerse reglas para validar los documentos.



VENTAJAS Y DESVENTAJAS



VENTAJAS



Modelo de datos flexible.



Los documentos pueden representar estructuras complejas.



Permite almacenar documentos relacionados mediante estructuras embebidas.



Facilita la evolución del modelo de datos.



Cuenta con capacidades de consulta e indexación.



DESVENTAJAS



La flexibilidad del esquema puede provocar inconsistencias si no se establecen reglas adecuadas.



Algunos modelos de datos pueden requerir un diseño cuidadoso para evitar consultas complejas.



No todos los problemas de datos son adecuados para un modelo documental.



Las relaciones altamente complejas pueden ser más naturales en una base de datos relacional.

CASOS DE USO

- Se desarrollan aplicaciones web medianas o grandes.
- Existen múltiples vistas, formularios y funcionalidades.
- Se necesita una estructura organizada para equipos de desarrollo.
- Se desarrollan sistemas empresariales, administrativos o dashboards.
- Se requiere una SPA con bastante lógica del lado del cliente.

No usar cuando:

- El sistema depende principalmente de relaciones complejas entre muchas entidades.
- Se requieren numerosas operaciones basadas en joins relacionales.
- El modelo de datos es altamente estructurado, estable y encaja naturalmente con un modelo relacional.



CASOS DE APLICACIÓN

- Comercio electrónico.
- Servicios financieros.
- Telecomunicaciones.
- Retail.
- Salud.
- Entretenimiento.
- IoT.



QUE TAN COMÚN ES EL STACK

- MongoDB es una base de datos NoSQL bastante utilizada en aplicaciones web y sistemas que trabajan con datos semiestructurados.
- Su modelo de documentos permite almacenar información en estructuras JSON/BSON y se integra fácilmente con aplicaciones desarrolladas en diferentes lenguajes.
- Dentro del stack, MongoDB se encarga del almacenamiento, mientras que FastAPI/Python maneja la lógica del backend y utiliza la base de datos para consultar y modificar información.
- REST define la forma en que el backend expone sus recursos, mientras que JSON facilita el intercambio de datos entre el frontend y el backend.

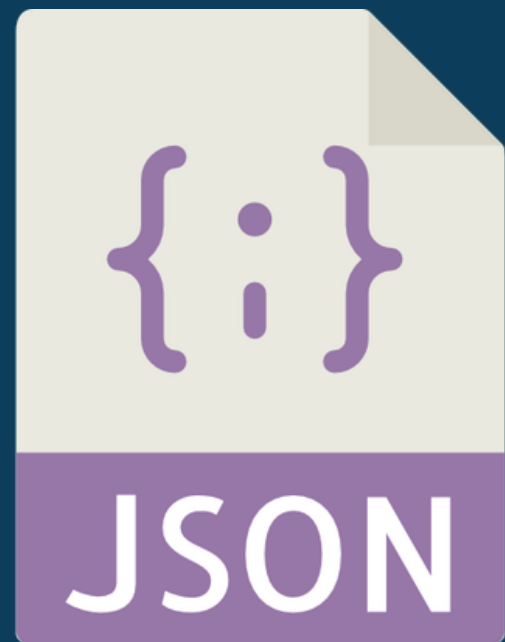


mongoDB®



SERVICIO, INTEGRACIÓN

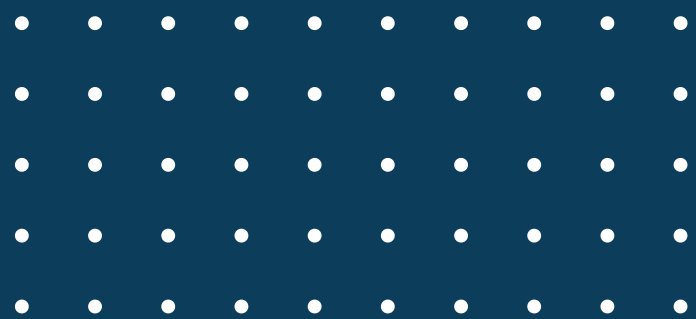
REST | JSON | API



- API (Application Programming Interface) - Una interfaz que permite que diferentes aplicaciones o componentes se comuniquen mediante operaciones y reglas definidas.
- REST (Representational State Transfer) - Un estilo arquitectónico para diseñar sistemas distribuidos.
- JSON (JavaScript Object Notation) - Un formato de intercambio de datos ligero, textual e independiente del lenguaje.

CARACTERÍSTICAS PRINCIPALES

- Separación cliente-servidor: el cliente y el servidor mantienen responsabilidades separadas.
- Comunicación mediante recursos: los recursos se identifican mediante URI.
- Métodos HTTP: utiliza operaciones como GET, POST, PUT y DELETE para expresar diferentes acciones.
- Stateless: cada solicitud debe contener la información necesaria para procesarla, sin depender de un estado almacenado en el servidor entre solicitudes.
- Representaciones: los recursos pueden representarse mediante diferentes formatos, como JSON.
- Interfaz uniforme: busca establecer una forma consistente de interactuar con los recursos.
- JSON: proporciona una forma sencilla y legible de representar los datos intercambiados entre cliente y servidor.



VENTAJAS Y DESVENTAJAS



VENTAJAS



Es relativamente sencillo de implementar y consumir.



Utiliza estándares ampliamente conocidos como HTTP.



Permite separar frontend y backend.



Facilita la integración entre tecnologías diferentes.



JSON es ligero, legible y compatible con numerosos lenguajes.



DESVENTAJAS



REST no define por sí mismo todos los detalles de una API concreta.



El diseño incorrecto de endpoints puede producir APIs difíciles de mantener.



Las respuestas JSON pueden contener información innecesaria si no se diseñan correctamente



Las aplicaciones pueden necesitar mecanismos adicionales para autenticación, autorización, versionamiento y control de errores.



Para comunicaciones que requieren características diferentes, otros estilos o protocolos pueden ser más apropiados

CASOS DE USO

- Un frontend necesita comunicarse con un backend.
- Una aplicación móvil necesita consumir servicios de un servidor.
- Diferentes sistemas necesitan intercambiar información.
- Se quiere exponer funcionalidades mediante una API.
- Se necesita integrar servicios externos.
- Se requiere una arquitectura donde cliente y servidor estén separados.

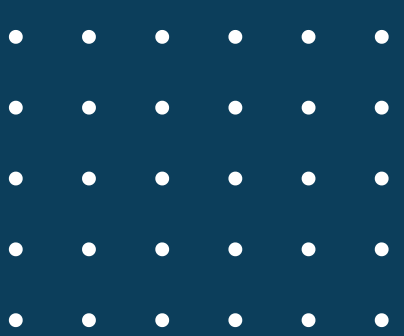
NO USAR CUANDO:

- La comunicación requiere principalmente tiempo real y actualización continua; pueden considerarse tecnologías como WebSockets.
- Se necesita una interfaz altamente especializada para consultas complejas; pueden considerarse otros enfoques como GraphQL.
- El sistema requiere comunicación interna con características específicas de rendimiento o serialización; pueden ser apropiados otros protocolos.



CASOS DE APLICACIÓN

- Aplicaciones web.
- Aplicaciones móviles.
- Sistemas empresariales.
- Servicios de comercio electrónico.
- Plataformas educativas.
- Servicios financieros.
- Sistemas de integración entre empresas.



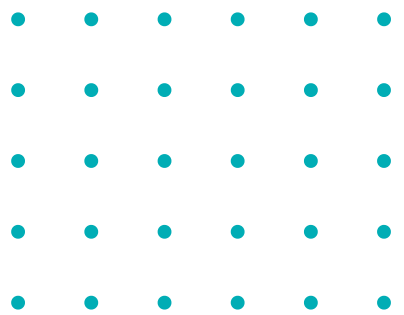
QUE TAN COMÚN ES EL STACK

El uso de APIs REST junto con JSON es muy común en aplicaciones web porque permite separar el frontend del backend y facilitar la comunicación entre diferentes componentes del sistema. REST define cómo se organizan y exponen los recursos, mientras que JSON es uno de los formatos utilizados para intercambiar los datos.

En el stack, Angular/JS consume la API REST, FastAPI/Python implementa los endpoints y procesa las solicitudes, JSON se utiliza para intercambiar la información y MongoDB almacena los datos que utiliza el backend.



DISEÑO DE EJEMPLO PRACTICO



HLD

DIAGRAMA DE ALTO NIVEL

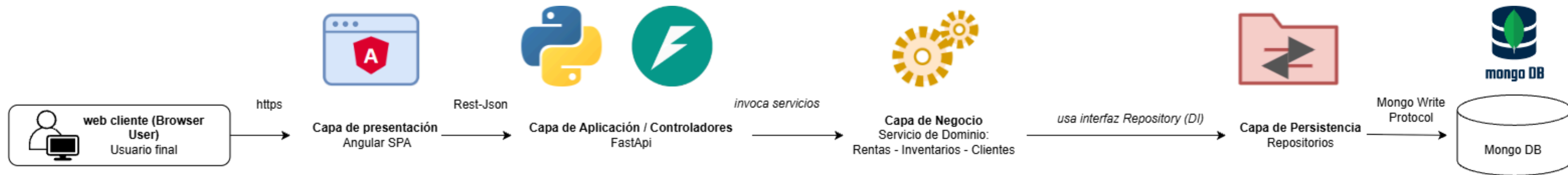


DIAGRAMA DE CONTEXTO

C4 NIVEL 1

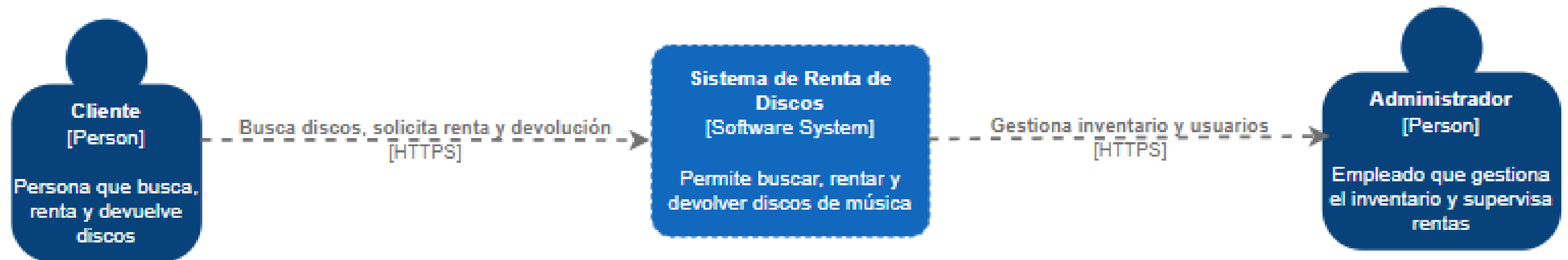
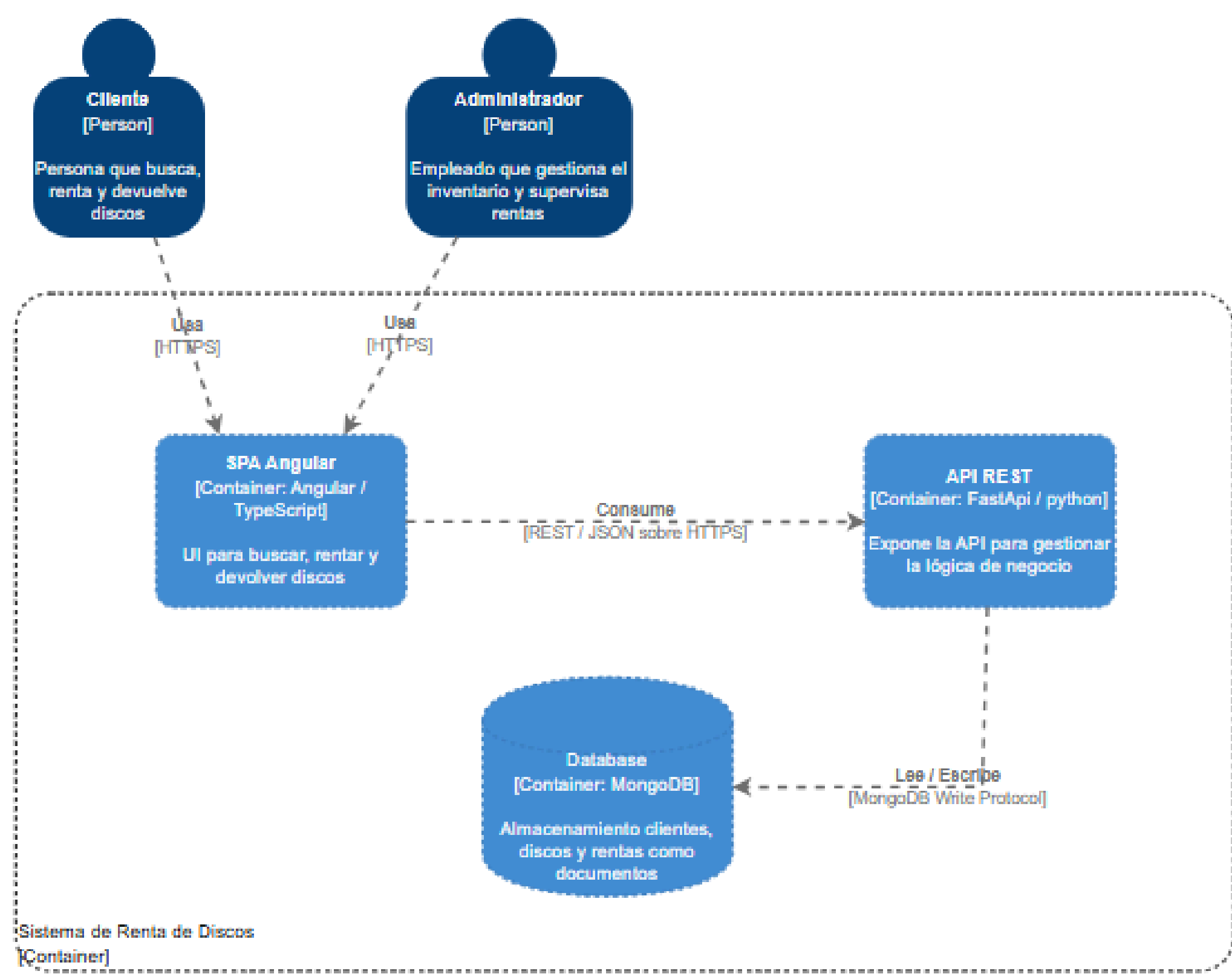
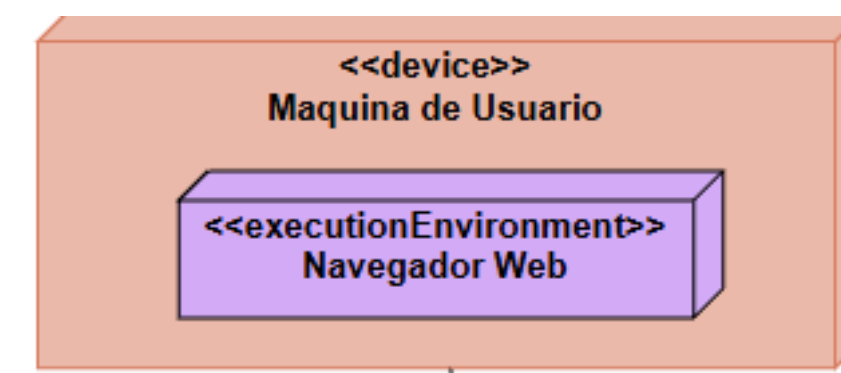


DIAGRAMA DE CONTENEDORES

C4 NIVEL 2





HTTPS/443

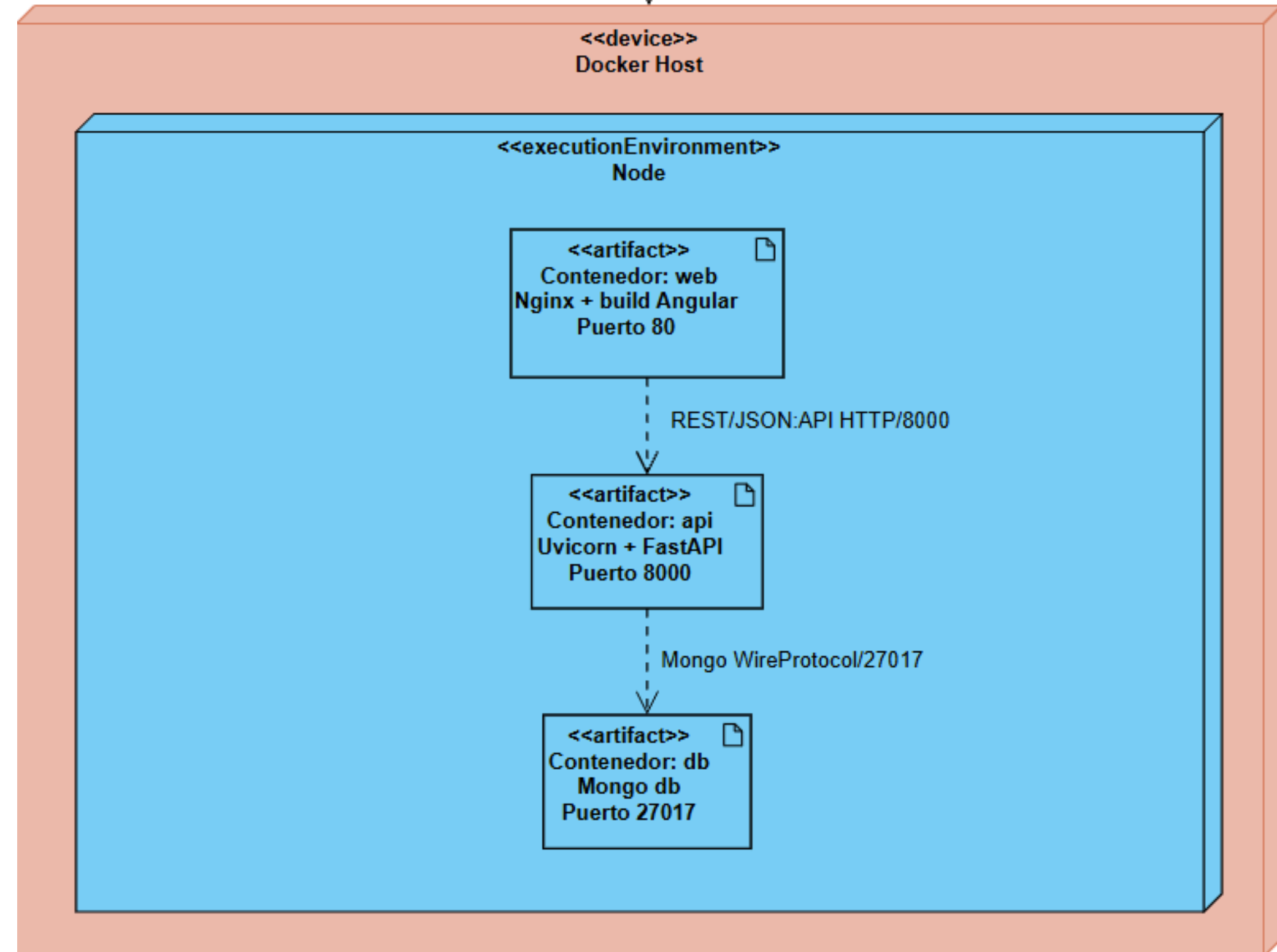
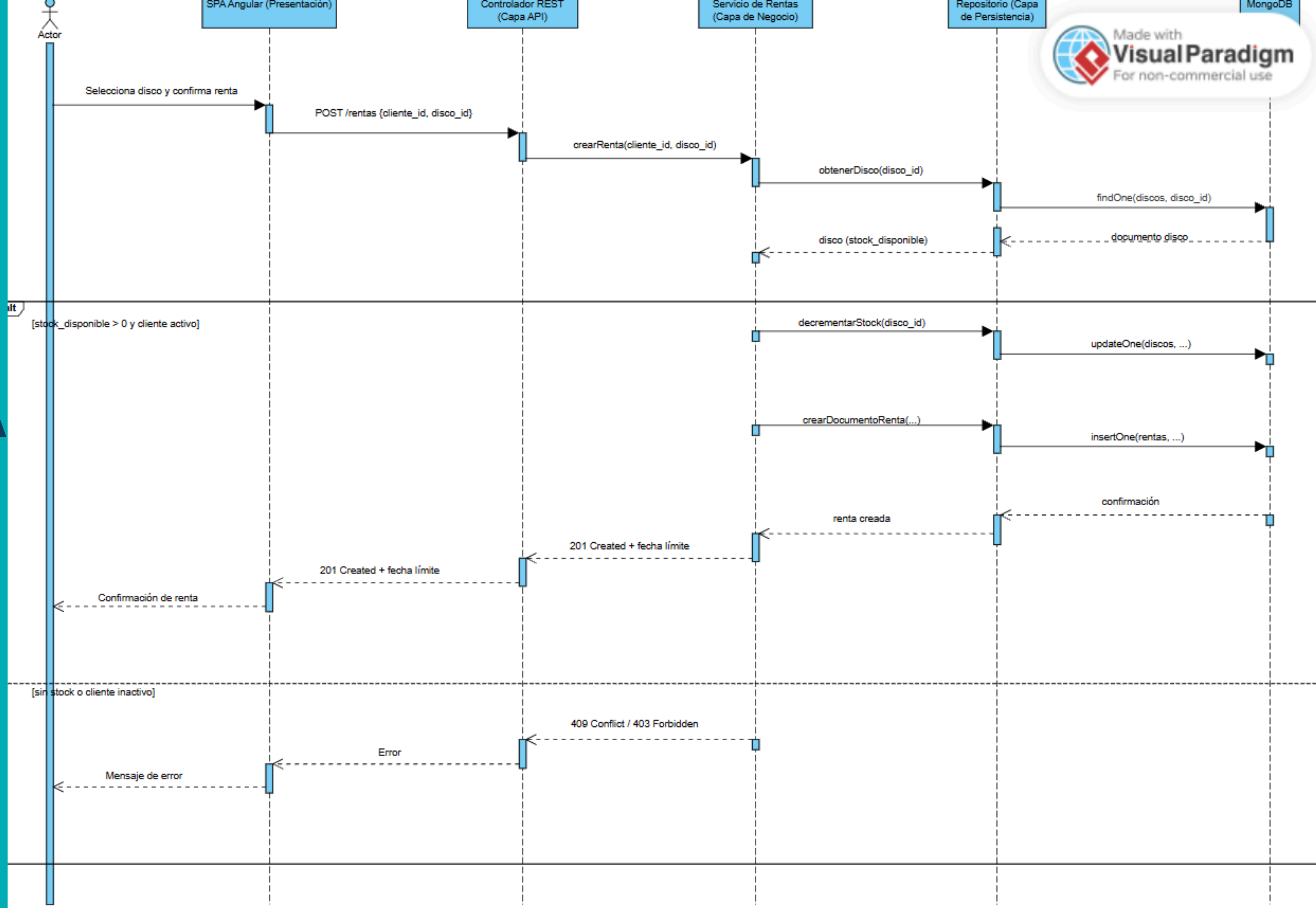


DIAGRAMA DE DESPLIEGUE



DIAGRAMA DINAMICO



LECCIONES APRENDIDAS

- Se comprende el funcionamiento de una arquitectura en capas, y que esta no define un número específico de estas, sirviendo como forma de estructurar proyectos de SW separando y definiendo las responsabilidades del programa en capas separadas..
- Se comprende el efecto de las capas separadas al momento de realizar cambios, pruebas y mantenimiento. Dónde patrones como Repository y Dependency Injection ayudan a evitar que una parte del sistema dependa demasiado de otra.
- Al analizar el stack, se comprende mejor el cómo cada tecnología cumple una función diferente dentro del sistema. Angular se encarga de la interfaz, FastAPI del backend, REST y JSON de la comunicación y MongoDB del almacenamiento de los datos.
- Finalmente, se reconoce como el elegir una tecnología no depende solamente de qué tan conocida o utilizada sea, sino de qué tan bien se adapta a las necesidades del sistema y a los problemas que se quieren resolver.

